



Lecture # 17-19 Functional Dependencies and Normalization for Relational Databases

Rashmi Dutta Baruah

Department of Computer Science & Engineering



भारतीय प्रौद्योगिकी संस्थान गुवाहाटी
Indian Institute of Technology Guwahati

Guwahati - 781039, INDIA





Outline

1 Informal Design Guidelines for Relational Databases

1.1 Semantics of the Relation Attributes

1.2 Redundant Information in Tuples and Update Anomalies

1.3 Null Values in Tuples

1.4 Spurious Tuples

2 Functional Dependencies (FDs)

2.1 Definition of FD

2.2 Inference Rules for FDs

2.3 Equivalence of Sets of FDs

2.4 Minimal Sets of FDs



Outline(contd.)

3 Normal Forms Based on Primary Keys

- 3.1 Normalization of Relations
- 3.2 Practical Use of Normal Forms
- 3.3 Definitions of Keys and Attributes Participating in Keys
- 3.4 First Normal Form
- 3.5 Second Normal Form
- 3.6 Third Normal Form

4 General Normal Form Definitions (For Multiple Keys)

5 BCNF (Boyce-Codd Normal Form)

6 Multivalued Dependency and Fourth Normal form

7 Join Dependency and Fifth Normal Form



Informal Design Guidelines for Relational Databases (1)

- What is relational database design?

The grouping of attributes to form "good" relation schemas

- Two levels of relation schemas
 - The logical "user view" level
 - The storage "base relation" level
- Design is concerned mainly with base relations
- What are the criteria for "good" base relations?



Informal Design Guidelines for Relational Databases (2)

- We first discuss informal guidelines for good relational design
- Then we discuss formal concepts of functional dependencies and normal forms
 - 1NF (First Normal Form)
 - 2NF (Second Normal Form)
 - 3NF (Third Normal Form)
 - BCNF (Boyce-Codd Normal Form)



1.1 Semantics of the Relation Attributes

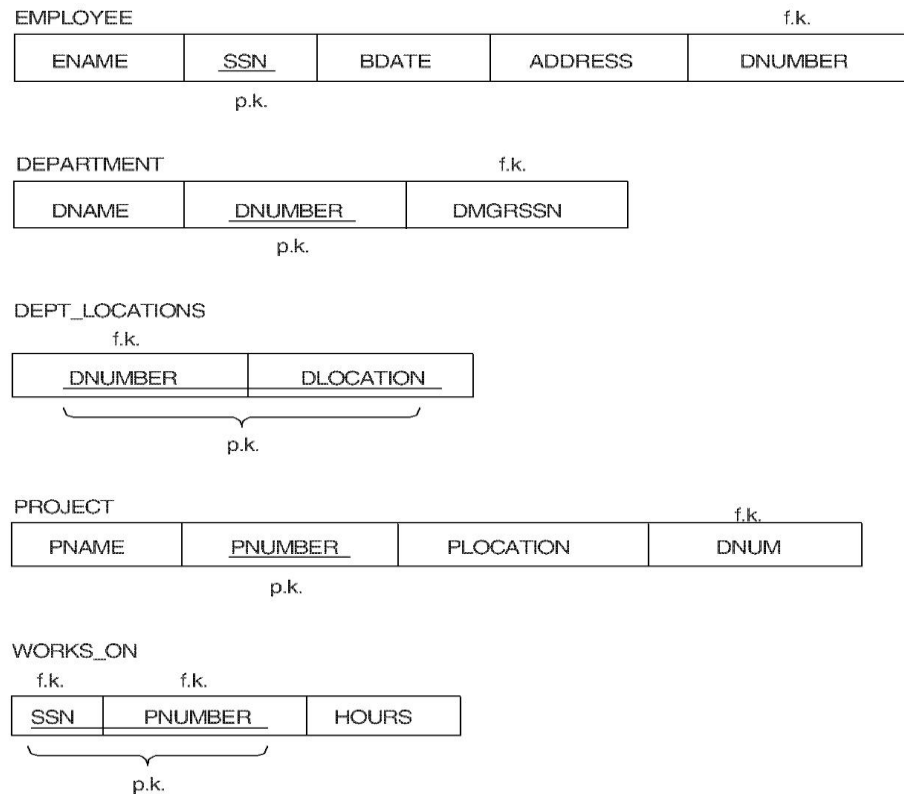
GUIDELINE 1: Informally, each tuple in a relation should represent one entity or relationship instance. (Applies to individual relations and their attributes).

- Attributes of different entities (EMPLOYEEs, DEPARTMENTs, PROJECTs) should not be mixed in the same relation
- Only foreign keys should be used to refer to other entities
- Entity and relationship attributes should be kept apart as much as possible.

Bottom Line: Design a schema that can be explained easily relation by relation. The semantics of attributes should be easy to interpret.

Figure: A simplified COMPANY relational database schema

Figure 14.1 Simplified version of the COMPANY relational database schema.





1.2 Redundant Information in Tuples and Update Anomalies

- Mixing attributes of multiple entities may cause problems
- Information is stored redundantly wasting storage
- Problems with update anomalies
 - Insertion anomalies
 - Deletion anomalies
 - Modification anomalies



EXAMPLE OF AN UPDATE ANOMALY (1)

Consider the relation:

EMP_PROJ (Emp#, Proj#, Ename, Pname, No_hours)

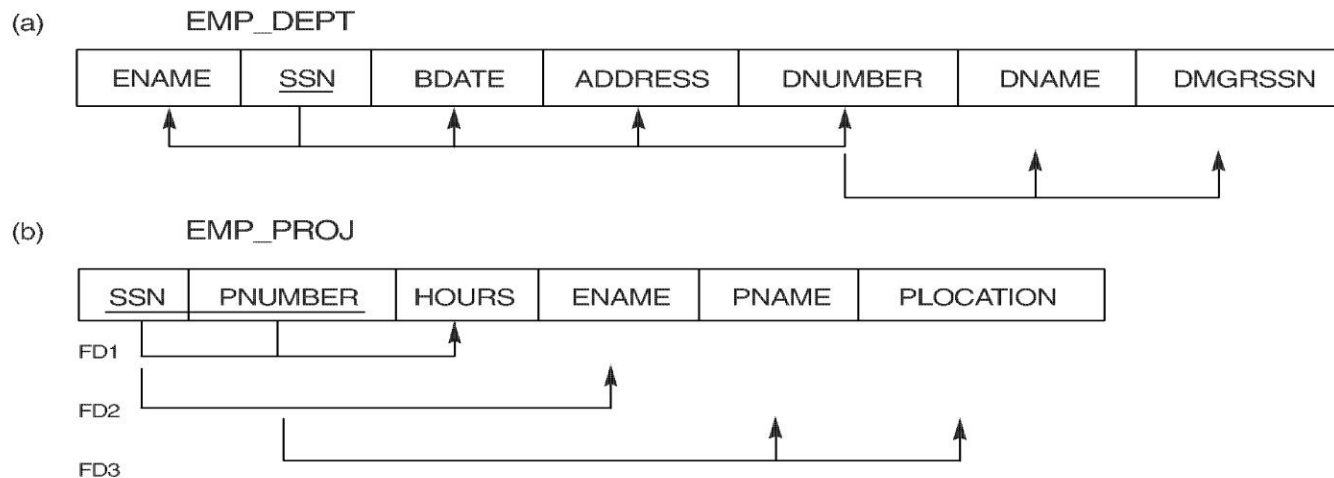
- **Update Anomaly:** Changing the name of project number P1 from “Billing” to “Customer-Accounting” may cause this update to be made for all 100 employees working on project P1.



EXAMPLE OF AN UPDATE ANOMALY (2)

- **Insert Anomaly:** Cannot insert a project unless an employee is assigned to .
Inversely - Cannot insert an employee unless an he/she is assigned to a project.
- **Delete Anomaly:** When a project is deleted, it will result in deleting all the employees who work on that project. Alternately, if an employee is the sole employee on a project, deleting that employee would result in deleting the corresponding project.

Figure: Two relation schemas suffering from update anomalies



© Addison Wesley Longman, Inc. 2000, Elmasri/Navathe, Fundamentals of Database Systems, Third Edition

Figure: Example States for EMP_DEPT and EMP_PROJ

EMP_DEPT						
ENAME	SSN	BDATE	ADDRESS	DNUMBER	DNAME	DMGRSSN
Smith,John B.	123456789	1965-01-09	731 Fondren,Houston,TX	5	Research	333445555
Wong,Franklin T.	333445555	1955-12-08	638 Voss,Houston,TX	5	Research	333445555
Zelaya,Alicia J.	999887777	1968-07-19	3321 Castle,Spring,TX	4	Administration	987654321
Wallace,Jennifer S.	987654321	1941-06-20	291 Berry,Bellaire,TX	4	Administration	987654321
Narayan,Ramesh K.	666884444	1962-09-15	975 FireOak,Humble,TX	5	Research	333445555
English,Joyce A.	453453453	1972-07-31	5631 Rice,Houston,TX	5	Research	333445555
Jabbar,Ahmad V.	987987987	1969-03-29	980 Dallas,Houston,TX	4	Administration	987654321
Borg,James E.	888665555	1937-11-10	450 Stone,Houston,TX	1	Headquarters	888665555

EMP_PROJ					
SSN	PNUMBER	HOURS	ENAME	PNAME	PLOCATION
123456789	1	32.5	Smith,John B.	ProductX	Bellaire
123456789	2	7.5	Smith,John B.	ProductY	Sugarland
666884444	3	40.0	Narayan,Ramesh K.	ProductZ	Houston
453453453	1	20.0	English,Joyce A.	ProductX	Bellaire
453453453	2	20.0	English,Joyce A.	ProductY	Sugarland
333445555	2	10.0	Wong,Franklin T.	ProductY	Sugarland
333445555	3	10.0	Wong,Franklin T.	ProductZ	Houston
333445555	10	10.0	Wong,Franklin T.	Computerization	Stafford
333445555	20	10.0	Wong,Franklin T.	Reorganization	Houston
999887777	30	30.0	Zelaya,Alicia J.	Newbenefits	Stafford
999887777	10	10.0	Zelaya,Alicia J.	Computerization	Stafford
987987987	10	35.0	Jabbar,Ahmad V.	Computerization	Stafford
987987987	30	5.0	Jabbar,Ahmad V.	Newbenefits	Stafford
987654321	30	20.0	Wallace,Jennifer S.	Newbenefits	Stafford
987654321	20	15.0	Wallace,Jennifer S.	Reorganization	Houston
888665555	20	null	Borg,James E.	Reorganization	Houston

© Addison Wesley Longman, Inc. 2000, Elmasri/Navathe, Fundamentals of Database Systems, Third Edition



Guideline to Redundant Information in Tuples and Update Anomalies

- **GUIDELINE 2:** Design a schema that does not suffer from the insertion, deletion and update anomalies. If there are any present, then note them so that applications can be made to take them into account



1.3 Null Values in Tuples

GUIDELINE 3: Relations should be designed such that their tuples will have as few NULL values as possible

- Attributes that are NULL frequently could be placed in separate relations (with the primary key)
- Reasons for nulls:
 - attribute not applicable or invalid
 - attribute value unknown (may exist)
 - value known to exist, but unavailable



1.4 Spurious Tuples

- Bad designs for a relational database may result in erroneous results for certain JOIN operations
- The "lossless join" property is used to guarantee meaningful results for join operations

GUIDELINE 4: The relations should be designed to satisfy the lossless join condition. No spurious tuples should be generated by doing a natural-join of any relations.

(a)

EMP_LOCS

<u>Ename</u>	<u>Plocation</u>
--------------	------------------

P.K.

EMP_PROJ1

<u>Ssn</u>	<u>Pnumber</u>	Hours	Pname	Plocation
------------	----------------	-------	-------	-----------

P.K.

(b)

EMP_LOCS

Ename	Plocation
Smith, John B.	Bellaire
Smith, John B.	Sugarland
Narayan, Ramesh K.	Houston
English, Joyce A.	Bellaire
English, Joyce A.	Sugarland
Wong, Franklin T.	Sugarland
Wong, Franklin T.	Houston
Wong, Franklin T.	Stafford
Zelaya, Alicia J.	Stafford
Jabbar, Ahmad V.	Stafford
Wallace, Jennifer S.	Stafford
Wallace, Jennifer S.	Houston
Borg, James E.	Houston

EMP_PROJ1

Ssn	Pnumber	Hours	Pname	Plocation
123456789	1	32.5	ProductX	Bellaire
123456789	2	7.5	ProductY	Sugarland
666884444	3	40.0	ProductZ	Houston
453453453	1	20.0	ProductX	Bellaire
453453453	2	20.0	ProductY	Sugarland
333445555	2	10.0	ProductY	Sugarland
333445555	3	10.0	ProductZ	Houston
333445555	10	10.0	Computerization	Stafford
333445555	20	10.0	Reorganization	Houston
999887777	30	30.0	Newbenefits	Stafford
999887777	10	10.0	Computerization	Stafford
987987987	10	35.0	Computerization	Stafford
987987987	30	5.0	Newbenefits	Stafford
987654321	30	20.0	Newbenefits	Stafford
987654321	20	15.0	Reorganization	Houston
888665555	20	NULL	Reorganization	Houston

Figure 14.5

Particularly poor design for the EMP_PROJ relation in Figure 14.3(b). (a) The two relation schemas EMP_LOCS and EMP_PROJ1. (b) The result of projecting the extension of EMP_PROJ from Figure 14.4 onto the relations EMP_LOCS and EMP_PROJ1.



2.1 Functional Dependencies (1)

- Functional dependencies (FDs) are used to specify *formal measures* of the "goodness" of relational designs
- FDs and keys are used to define **normal forms** for relations
- FDs are **constraints** that are derived from the *meaning* and *interrelationships* of the data attributes
- A set of attributes *X functionally determines* a set of attributes *Y* if the value of *X* determines a unique value for *Y*



Functional Dependencies (2)

- $X \rightarrow Y$ holds if whenever two tuples have the same value for X , they *must have* the same value for Y
- For any two tuples $t1$ and $t2$ in any relation instance $r(R)$: *If* $t1[X]=t2[X]$, *then* $t1[Y]=t2[Y]$
- $X \rightarrow Y$ in R specifies a *constraint* on all relation instances $r(R)$
- Written as $X \rightarrow Y$; can be displayed graphically on a relation schema as in Figures. (denoted by the arrow:).
- FDs are derived from the real-world constraints on the attributes



Examples of FD constraints (1)

- social security number determines employee name
SSN $\rightarrow$ ENAME
- project number determines project name and location
PNUMBER $\rightarrow$ {PNAME, PLOCATION}
- employee ssn and project number determines the hours per week that the employee works on the project
{SSN, PNUMBER} $\rightarrow$ HOURS



Examples of FD constraints (2)

- An FD is a property of the attributes in the schema R .
- The constraint must hold on *every relation instance* $r(R)$
- If K is a key of R , then K functionally determines all attributes in R (since we never have two distinct tuples with $t1[K]=t2[K]$)



2.2 Inference Rules for FDs (1)

- Given a set of FDs F , we can *infer* additional FDs that hold whenever the FDs in F hold

Armstrong's inference rules:

IR1. (**Reflexive**) If Y subset-of X , then $X \rightarrow Y$

IR2. (**Augmentation**) If $X \rightarrow Y$, then $XZ \rightarrow YZ$

(Notation: XZ stands for $X \cup Z$)

IR3. (**Transitive**) If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

- IR1, IR2, IR3 form a *sound* and *complete* set of inference rules



Inference Rules for FDs (2)

Some **additional inference rules** that are useful:

(Decomposition) If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$

(Union) If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$

(Pseudotransitivity) If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$

- The last three inference rules, as well as any other inference rules, can be deduced from IR1, IR2, and IR3 (completeness property)



Inference Rules for FDs (3)

- **Closure** of a set F of FDs is the set F^+ of all FDs that can be inferred from F
- **Closure** of a set of attributes X with respect to F is the set X^+ of all attributes that are functionally determined by X
- X^+ can be calculated by repeatedly applying IR1, IR2, IR3 using the FDs in F

Algorithm. Determining X^+ , the Closure of X under F

Input: A set F of FDs on a relation schema R , and a set of attributes X , which is a subset of R .

$X^+ := X$;

repeat

$\text{old}X^+ := X^+$;

 for each functional dependency $Y \rightarrow Z$ in F do

 if Y is subset of X^+ then $X^+ := X^+ \cup Z$;

until ($X^+ = \text{old}X^+$);



2.3 Equivalence of Sets of FDs

- Two sets of FDs F and G are **equivalent** if:
 - every FD in F can be inferred from G , *and*
 - every FD in G can be inferred from F
- Hence, F and G are equivalent if $F^+ = G^+$

Definition: F **covers** G if every FD in G can be inferred from F (i.e., if $G^+ \text{ subset-of } F^+$)

- F and G are equivalent if F covers G and G covers F



2.4 Minimal Sets of FDs (1)

- A set of FDs is **minimal** if it satisfies the following conditions:
 - (1) Every dependency in F has a single attribute for its RHS.
 - (2) We cannot remove any dependency from F and have a set of dependencies that is equivalent to F .
 - (3) We cannot replace any dependency $X \rightarrow A$ in F with a dependency $Y \rightarrow A$, where Y proper-subset-of X and still have a set of dependencies that is equivalent to F .
- Every set of FDs has an equivalent minimal set
- There can be several equivalent minimal sets



Algorithm Finding a Minimal Cover F for a Set of Functional Dependencies E

Input: A set of functional dependencies E .

1. Set $F := E$.
2. Replace each functional dependency $X \rightarrow \{A_1, A_2, \dots, A_n\}$ in F by the n functional dependencies $X \rightarrow A_1, X \rightarrow A_2, \dots, X \rightarrow A_n$. (*This places the FDs in a canonical form for subsequent testing*)
3. For each functional dependency $X \rightarrow A$ in F
 for each attribute B that is an element of X
 if $\{F - \{X \rightarrow A\}\} \cup \{(X - \{B\}) \rightarrow A\}$ is equivalent to F
 then replace $X \rightarrow A$ with $(X - \{B\}) \rightarrow A$ in F .
(*This constitutes removal of an extraneous attribute B contained in the lefthand side X of a functional dependency $X \rightarrow A$ when possible*)
4. For each remaining functional dependency $X \rightarrow A$ in F
 if $\{F - \{X \rightarrow A\}\}$ is equivalent to F ,
 then remove $X \rightarrow A$ from F . (*This constitutes removal of a redundant functional dependency $X \rightarrow A$ from F when possible*)



3 Normal Forms Based on Primary Keys

3.1 Normalization of Relations

3.2 Practical Use of Normal Forms

3.3 Definitions of Keys and Attributes
Participating in Keys

3.4 First Normal Form

3.5 Second Normal Form

3.6 Third Normal Form



3.1 Normalization of Relations (1)

- **Normalization:** The process of decomposing unsatisfactory "bad" relations by breaking up their attributes into smaller relations
- **Normal form:** Condition using keys and FDs of a relation to certify whether a relation schema is in a particular normal form



Normalization of Relations (2)

- 2NF, 3NF, BCNF based on keys and FDs of a relation schema
- 4NF based on keys, multi-valued dependencies : MVDs; 5NF based on keys, join dependencies : JDs
- Additional properties may be needed to ensure a good relational design (lossless join, dependency preservation)



3.2 Practical Use of Normal Forms

- **Normalization** is carried out in practice so that the resulting designs are of high quality and meet the desirable properties
- The practical utility of these normal forms becomes questionable when the constraints on which they are based are **hard to understand** or to **detect**
- The database designers ***need not*** normalize to the highest possible normal form. (usually up to 3NF, BCNF or 4NF)
- **Denormalization:** the process of storing the join of higher normal form relations as a base relation—which is in a lower normal form



3.3 Definitions of Keys and Attributes Participating in Keys (1)

- A **superkey** of a relation schema $R = \{A_1, A_2, \dots, A_n\}$ is a set of attributes S subset-of R with the property that no two tuples t_1 and t_2 in any legal relation state r of R will have $t_1[S] = t_2[S]$
- A **key** K is a superkey with the *additional property* that removal of any attribute from K will cause K not to be a superkey any more.



Definitions of Keys and Attributes Participating in Keys (2)

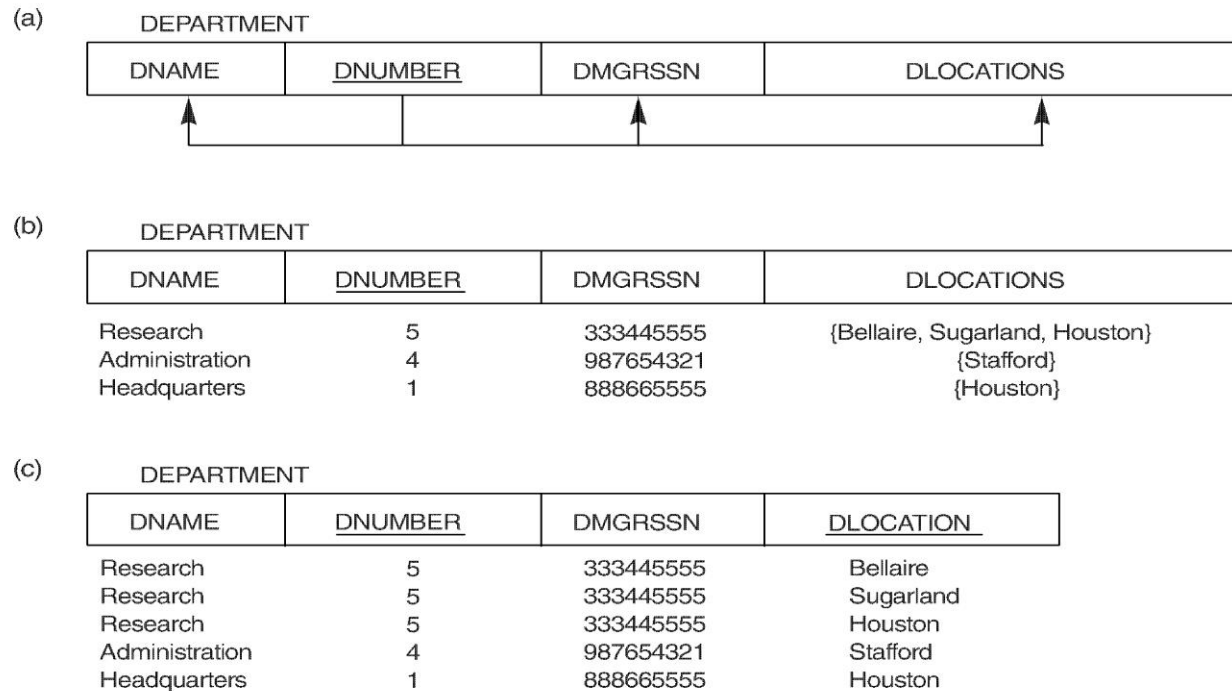
- If a relation schema has more than one key, each is called a **candidate key**. One of the candidate keys is *arbitrarily* designated to be the **primary key**, and the others are called *secondary keys*.
- A **Prime attribute** must be a member of *some candidate key*
- A **Nonprime attribute** is not a prime attribute—that is, it is not a member of any candidate key.



3.4 First Normal Form

- Disallows composite attributes, multivalued attributes, and **nested relations**; attributes whose values *for an individual tuple* are non-atomic.
- Three techniques to achieve first normal form for relations with multivalued attributes:
 - Remove the attribute that violates 1 NF and place it in a separate locations, primary key will be this attribute and the pk of the original relation
 - Expand the key by including the multivalued attribute
 - If maximum number of values is know then replace the attribute by that many number of atomic attributes.

Figure: Normalization into 1NF



© Addison Wesley Longman, Inc. 2000, Elmasri/Navathe, Fundamentals of Database Systems, Third Edition

Note: The above figure is now called Figure 10.8 in Edition 4

Figure: Normalization nested relations into 1NF

(a)

EMP_PROJ

SSN	ENAME	PROJS	
		PNUMBER	HOURS

(b)

EMP_PROJ

SSN	ENAME	PNUMBER	HOURS
123456789	Smith,John B.	1	32.5
		2	7.5
666884444	Narayan,Ramesh K.	3	40.0
453453453	English,Joyce A.	1	20.0
		2	20.0
333445555	Wong,Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya,Alicia J.	30	30.0
		10	10.0
987987987	Jabbar,Ahmad V.	10	35.0
		30	5.0
987654321	Wallace,Jennifer S.	30	20.0
		20	15.0
888665555	Borg,James E.	20	null

(c)

EMP_PROJ1

<u>SSN</u>	ENAME
------------	-------

EMP_PROJ2

<u>SSN</u>	<u>PNUMBER</u>	HOURS
------------	----------------	-------



3.5 Second Normal Form (1)

- Uses the concepts of **FDs**, **primary key**

Definitions:

- **Prime attribute** - attribute that is member of the primary key K
- **Full functional dependency** - a FD $Y \rightarrow Z$ where removal of any attribute from Y means the FD does not hold any more

Examples:

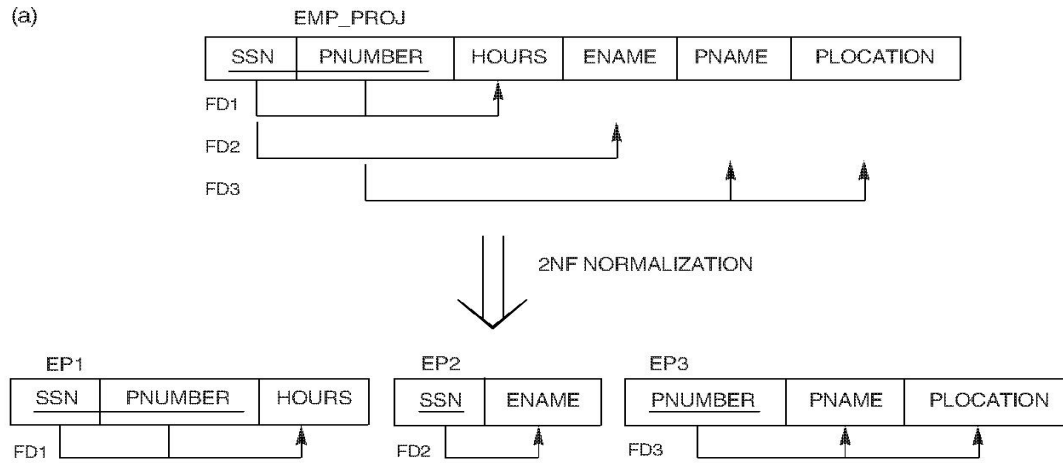
- $\{SSN, PNUMBER\} \rightarrow HOURS$ is a full FD since neither $SSN \rightarrow HOURS$ nor $PNUMBER \rightarrow HOURS$ hold
- $\{SSN, PNUMBER\} \rightarrow ENAME$ is *not* a full FD (it is called a *partial dependency*) since $SSN \rightarrow ENAME$ also holds



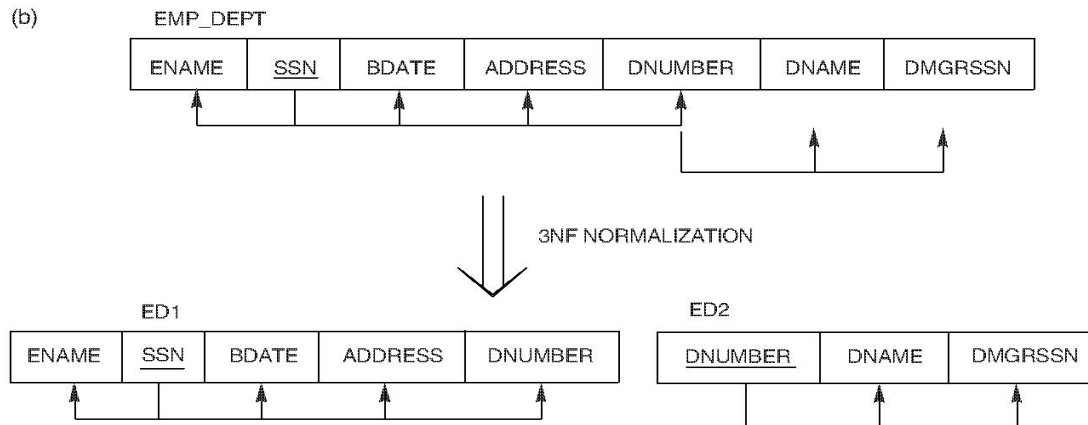
Second Normal Form (2)

- A relation schema R is in **second normal form (2NF)** if every non-prime attribute A in R is fully functionally dependent on the primary key
- R can be decomposed into 2NF relations via the process of 2NF normalization

(a)



(b)





3.6 Third Normal Form (1)

Definition:

- **Transitive functional dependency** - a FD $X \rightarrow Z$ that can be derived from two FDs $X \rightarrow Y$ and $Y \rightarrow Z$

Examples:

- $SSN \rightarrow DMGRSSN$ is a *transitive* FD since $SSN \rightarrow DNUMBER$ and $DNUMBER \rightarrow DMGRSSN$ hold
- $SSN \rightarrow ENAME$ is *non-transitive* since there is no set of attributes X where $SSN \rightarrow X$ and $X \rightarrow ENAME$



Third Normal Form (2)

- A relation schema R is in **third normal form (3NF)** if it is in 2NF *and* no non-prime attribute A in R is transitively dependent on the primary key
- R can be decomposed into 3NF relations via the process of 3NF normalization

NOTE:

In $X \rightarrow Y$ and $Y \rightarrow Z$, with X as the primary key, we consider this a problem only if Y is not a candidate key. When Y is a candidate key, there is no problem with the transitive dependency .

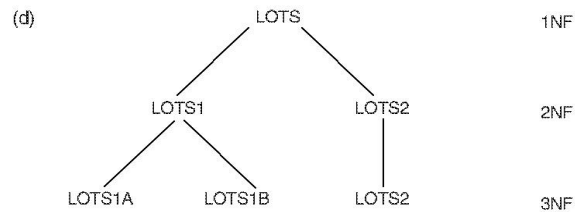
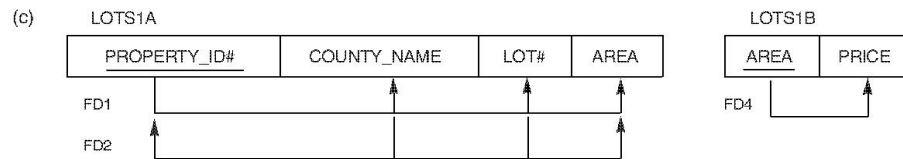
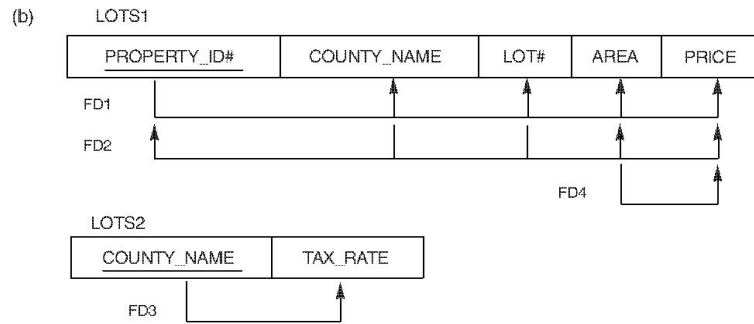
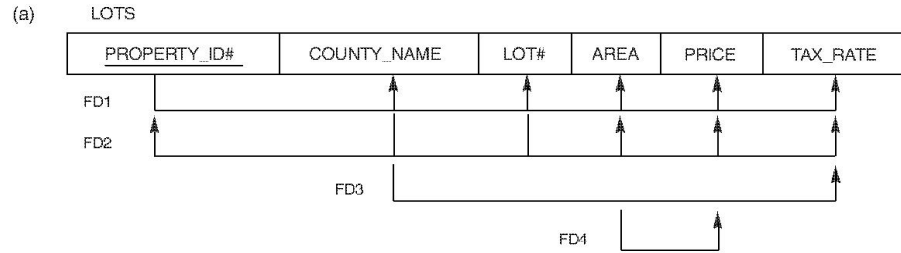
E.g., Consider EMP (SSN, Emp#, Salary).

Here, $SSN \rightarrow Emp\# \rightarrow Salary$ and Emp# is a candidate key.



4 General Normal Form Definitions (For Multiple Keys) (1)

- The previous definitions consider the primary key only
- The following more general definitions take into account relations with multiple candidate keys
- A relation schema R is in **second normal form (2NF)** if every non-prime attribute A in R is fully functionally dependent on *every key* of R





4 General Normal Form Definitions (2)

Definition:

- **Superkey** of relation schema R - a set of attributes S of R that contains a key of R
- A relation schema R is in **third normal form (3NF)** if whenever a FD $X \rightarrow A$ holds in R, then either:
 - (a) X is a superkey of R, or
 - (b) A is a prime attribute of R

NOTE: Boyce-Codd normal form disallows condition (b) above



5 BCNF (Boyce-Codd Normal Form)

- A relation schema R is in **Boyce-Codd Normal Form (BCNF)** if whenever an FD $X \rightarrow A$ holds in R , then X is a superkey of R
- Each normal form is strictly stronger than the previous one
 - Every 2NF relation is in 1NF
 - Every 3NF relation is in 2NF
 - Every BCNF relation is in 3NF
- There exist relations that are in 3NF but not in BCNF
- The goal is to have each relation in BCNF (or 3NF)

Figure 14.12 Boyce-Codd normal form. (a) BCNF normalization with the dependency of FD2 being “lost” in the decomposition. (b) A relation R in 3NF but not in BCNF.

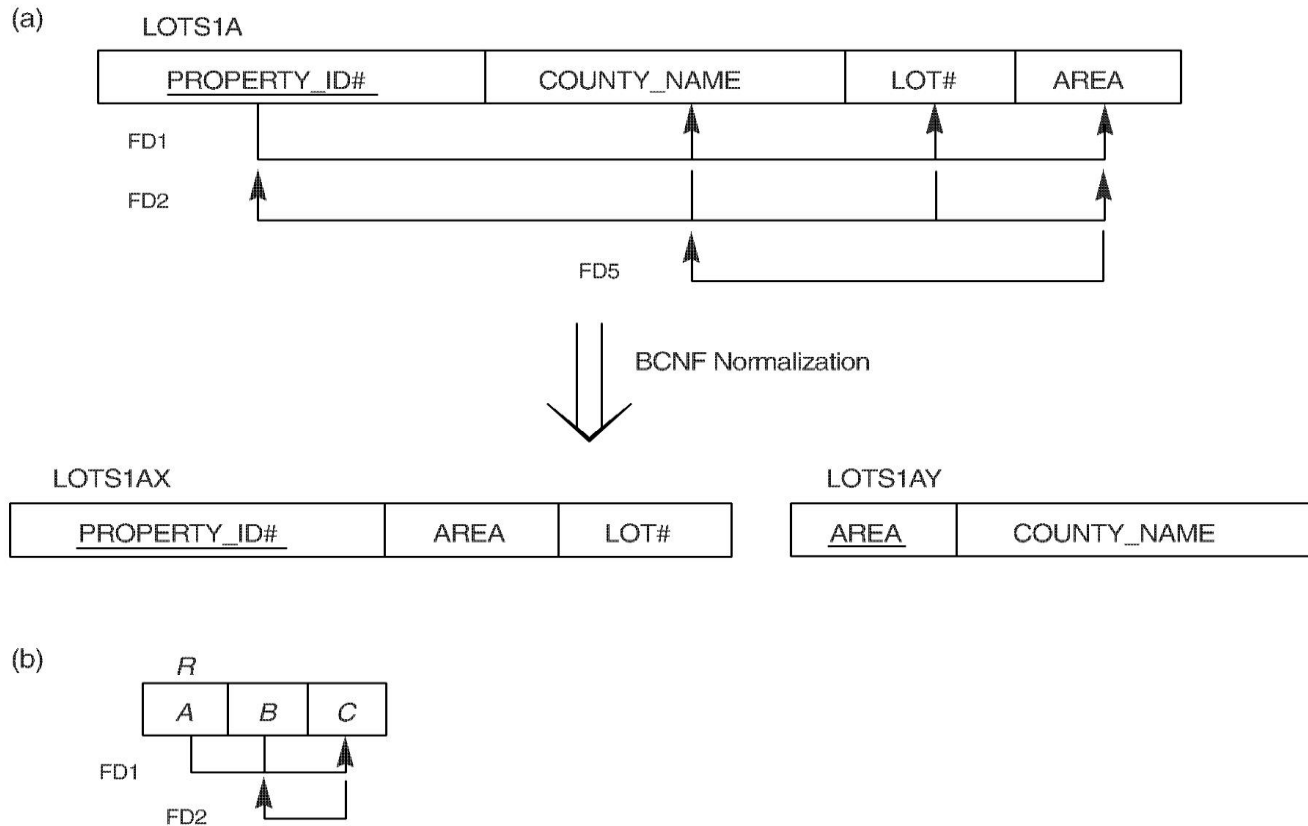




Figure: a relation TEACH that is in 3NF but not in BCNF

{Student, Course} is a candidate key for this relation.

TEACH

STUDENT	COURSE	INSTRUCTOR
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe

fd1: { student, course} $\rightarrow$ instructor, fd2: instructor $\rightarrow$ course



Achieving the BCNF by Decomposition (1)

- Two FDs exist in the relation TEACH:
fd1: { student, course} $\rightarrow$ instructor
fd2: instructor $\rightarrow$ course
- {student, course} is a candidate key for this relation and that the dependencies shown follow the pattern in Figure 14.12 (b). So this relation is in 3NF but not in BCNF
- A relation **NOT** in BCNF should be decomposed so as to meet this property, while possibly forgoing the preservation of all functional dependencies in the decomposed relations.



Achieving the BCNF by Decomposition (2)

- Three possible decompositions for relation TEACH
 1. {student, instructor} and {student, course}
 2. {course, instructor} and {course, student}
 3. {instructor, course} and {instructor, student}
- All three decompositions will lose fd1. We have to settle for sacrificing the functional dependency preservation. But we cannot sacrifice the non-additivity property after decomposition.



Achieving the BCNF by Decomposition (2)

- A test to determine whether a binary decomposition (decomposition into two relations) is nonadditive (lossless) is:
- **NJB (Non-additive Join test for Binary decomposition):** A decomposition $D = \{R_1, R_2\}$ of R has the lossless (nonadditive) join property with respect to a set of functional dependencies F on R iff either
 - The FD $((R_1 \cap R_2) \rightarrow (R_1 - R_2))$ is in F^+ **or**
 - The FD $((R_1 \cap R_2) \rightarrow (R_2 - R_1))$ is in F^+



Achieving the BCNF by Decomposition (2)

- Out of the previous three, only the 3rd decomposition will not generate spurious tuples after join.(and hence has the non-additivity property).

{instructor, course } and {instructor, student}

- In general, a relation R not in BCNF can be decomposed so as to meet the nonadditive join property. It decomposes R successively into a set of relations that are in BCNF.
 - Let R be the relation not in BCNF, let $X \subseteq R$, and let $X \rightarrow A$ be the FD that causes a violation of BCNF. R may be decomposed into two relations:
 - **R-A**
 - **XA**
 - If either R-A or XA, is not in BCNF, repeat the process.

6. Multivalued Dependencies and Fourth Normal Form (1)

(a) The EMP relation with two MVDs: $\text{ENAME} \twoheadrightarrow \text{PNAME}$ and $\text{ENAME} \twoheadrightarrow \text{DNAME}$. (b) Decomposing the EMP relation into two 4NF relations EMP_PROJECTS and EMP_DEPENDENTS.

(a) **EMP**

<u>ENAME</u>	PNAME	<u>DNAME</u>
--------------	-------	--------------

Smith	X	John
Smith	Y	Anna
Smith	X	Anna
Smith	Y	John

(b) **EMP_PROJECTS**

<u>ENAME</u>	<u>PNAME</u>
--------------	--------------

Smith	X
Smith	Y

EMP_DEPENDENTS

<u>ENAME</u>	<u>DNAME</u>
--------------	--------------

Smith	John
Smith	Anna



Multivalued Dependencies and Fourth Normal Form (2)

Definition:

- A **multivalued dependency (MVD)** $X \twoheadrightarrow Y$ specified on relation schema R , where X and Y are both subsets of R , specifies the following constraint on any relation state r of R : If two tuples t_1 and t_2 exist in r such that $t_1[X] = t_2[X]$, then two tuples t_3 and t_4 should also exist in r with the following properties, where we use Z to denote $(R - (X \cup Y))$:
 - $t_3[X] = t_4[X] = t_1[X] = t_2[X]$.
 - $t_3[Y] = t_1[Y]$ and $t_4[Y] = t_2[Y]$.
 - $t_3[Z] = t_2[Z]$ and $t_4[Z] = t_1[Z]$.
- An MVD $X \twoheadrightarrow Y$ in R is called a **trivial MVD** if (a) Y is a subset of X , or (b) $X \cup Y = R$.



Multivalued Dependencies and Fourth Normal Form (3)

Definition:

- A relation schema R is in **4NF** with respect to a set of dependencies F (that includes functional dependencies and multivalued dependencies) if, for **every nontrivial multivalued dependency $X \twoheadrightarrow Y$ in F^+** , X is a superkey for R .

F^+ refers to the cover of functional dependencies F , or all dependencies that are implied by F .

Multivalued Dependencies and Fourth Normal Form (4)

Decomposing a relation state of EMP that is not in 4NF. (a) EMP relation with additional tuples. (b) Two corresponding 4NF relations EMP_PROJECTS and EMP_DEPENDENTS.

(a) **EMP**

<u>ENAME</u>	PNAME	<u>DNAME</u>
--------------	-------	--------------

Smith	X	John
Smith	Y	Anna
Smith	X	Anna
Smith	Y	John
Brown	W	Jim
Brown	X	Jim
Brown	Y	Jim
Brown	Z	Jim
Brown	W	Joan
Brown	X	Joan
Brown	Y	Joan
Brown	Z	Joan
Brown	W	Bob
Brown	X	Bob
Brown	Y	Bob
Brown	Z	Bob

(b) **EMP_PROJECTS**

<u>ENAME</u>	<u>PNAME</u>
--------------	--------------

Smith	X
Smith	Y
Brown	W
Brown	X
Brown	Y
Brown	Z

EMP_DEPENDENTS

<u>ENAME</u>	<u>DNAME</u>
--------------	--------------

Smith	Anna
Smith	John
Brown	Jim
Brown	Joan
Brown	Bob



7. Join Dependencies and Fifth Normal Form (1)

Definition:

- A **join dependency (JD)**, denoted by $JD(R_1, R_2, \dots, R_n)$, specified on relation schema R , specifies a constraint on the states r of R . The constraint states that every legal state r of R should have a non-additive join decomposition into $R_1, R_2, \dots, R_n$; that is, for every such r we have

$$* (\pi_{R_1}(r), \pi_{R_2}(r), \dots, \pi_{R_n}(r)) = r$$

Note: an MVD is a special case of a JD where $n = 2$.

- A join dependency $JD(R_1, R_2, \dots, R_n)$, specified on relation schema R , is a **trivial JD** if one of the relation schemas R_i in $JD(R_1, R_2, \dots, R_n)$ is equal to R .



Join Dependencies and Fifth Normal Form (2)

Definition:

- A relation schema R is in **fifth normal form (5NF)** (or **Project-Join Normal Form (PJNF)**) with respect to a set F of functional, multivalued, and join dependencies if, for every nontrivial join dependency $JD(R_1, R_2, \dots, R_n)$ in F^+ (that is, implied by F), every R_i is a superkey of R .

Relation SUPPLY with Join Dependency and conversion to Fifth Normal Form

(c) The relation SUPPLY with no MVDs is in 4NF but not in 5NF if it has the JD(R1, R2, R3). (d) Decomposing the relation SUPPLY into the 5NF relations R1, R2, and R3.

(c) **SUPPLY**

SNAME	PARTNAME	PROJNAME
Smith	Bolt	ProjX
Smith	Nut	ProjY
Adamsky	Bolt	ProjY
Walton	Nut	ProjZ
Adamsky	Nail	ProjX
Adamsky	Bolt	ProjX
Smith	Bolt	ProjY

(d) **R1**

SNAME	PARTNAME
Smith	Bolt
Smith	Nut
Adamsky	Bolt
Walton	Nut
Adamsky	Nail

R2

SNAME	PROJNAME
Smith	ProjX
Smith	ProjY
Adamsky	ProjY
Walton	ProjZ
Adamsky	ProjX

R3

PARTNAME	PROJNAME
Bolt	ProjX
Nut	ProjY
Bolt	ProjY
Nut	ProjZ
Nail	ProjX



Summary

- Identified informally some of the measures for indicating whether a relation schema is good or bad.
- Discussed informal guidelines for a good design.
- Presented some formal concepts that allow relational design in a top-down fashion by analysing relations individually. We defined this process of design by analysis and decomposition by introducing the process of normalization.
- The concept of functional dependency is defined some of its properties are discussed.
- Described the normalization process of achieving good designs.
- Discussed 1NF, 2 NF, 3NF, BCNF and 4 NF with examples